

QUIZ SYLLABUS

→ until MSTs
(10:15 - 11:15)

- HUFFMAN CODES:

$$\left. \begin{array}{l} A-Z \\ a-z \\ 0-9 \end{array} \right\} \begin{array}{l} 52 \\ 10 \end{array} \right\} 62$$

With $\lceil \log n \rceil$ many bits, we can represent n distinct letters. (each letter $\lceil \log n \rceil$ bits)

Consider the English language.

$$1\,000\,000 \text{ letters} \Rightarrow (1\,000\,000) \times \lceil \log n \rceil \text{ bits.}$$

We want an encoding such that for any pair of letters u, v :

- $\text{Enc}(u)$ is not a prefix of $\text{Enc}(v)$
- $\text{Enc}(v)$ " " $\text{Enc}(u)$

} "not prefix" property

- Trivial encoding using binary representation:

"Algorithm" $\rightarrow$ $\underbrace{\text{Enc}(A) \cdot \text{Enc}(I) \cdot \text{Enc}(g) \cdot \dots \cdot \text{Enc}(m)}_{\text{binary string}}$

Here, the length of Enc for all letters is the same. $\Rightarrow$ we don't need the not prefix property

Suppose we move to a scheme where the lengths are not all the same. Then, the "not prefix" property helps.



$a = 001, b = 01, c = 111$

Only works due to "not prefix" property.

Otherwise there would be ambiguity. Where to stop reading?

Letters	a	b	c	d	e
Frequencies	32	25	2	18	05

frequencies / probabilities

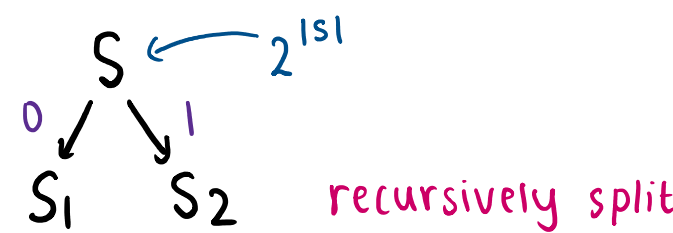
* TASK: Construct a scheme of encoding s.t.

1. Optimise the average bit length (ABL).
2. "not prefix" property holds.

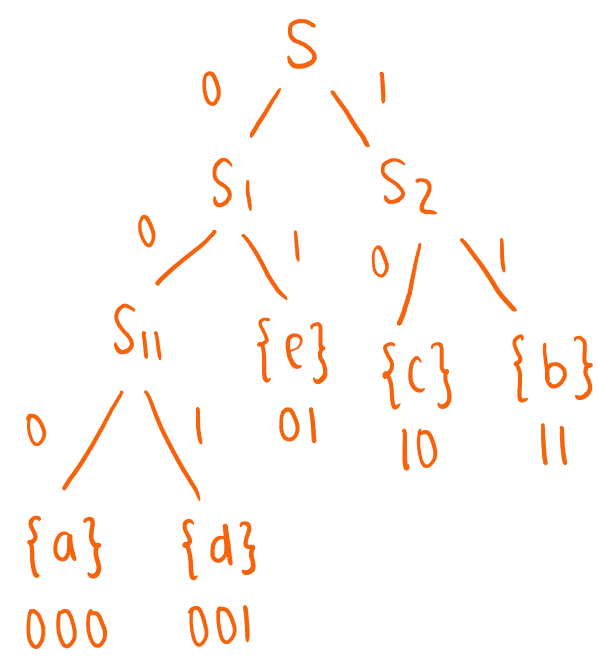
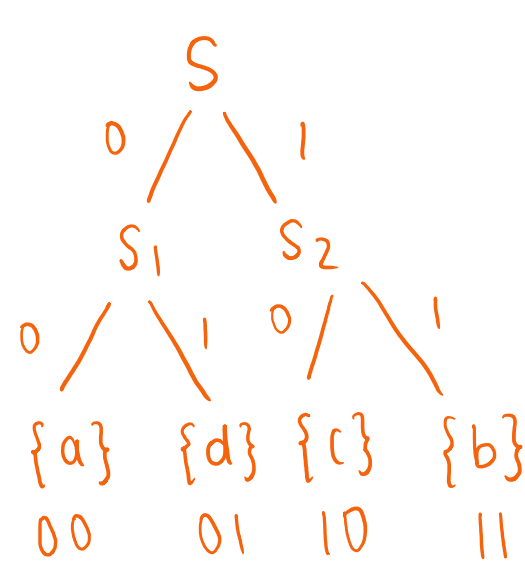
$$\begin{aligned} \text{ABL}(\text{text}) &= \sum_{\text{letter} \in \text{text}} \frac{(\# \text{ of occurrences of letter}) \cdot |\text{Enc}(\text{letter})|}{\text{total length of text}} \\ &= \sum_{\text{letter} \in \text{text}} \text{freq.}(\text{letter}) \cdot |\text{Enc}(\text{letter})| \end{aligned}$$

- SHANNON-FANO SCHEMES:

Let S be the set of letters.



divide s.t. total freq. of S_1
 $\cong$ total freq. of S_2



$$\begin{aligned} \text{freq}(a) + \text{freq}(d) &\approx \text{freq}(e) \\ \text{freq}(c) + \text{freq}(b) &\approx \text{freq}(a) + \text{freq}(d) + \text{freq}(e) \\ \text{freq}(a) &\approx \text{freq}(d) \end{aligned}$$

- no prefixes since we are only labelling leaf nodes.
- root to leaf path gives encoding

PROBLEMS:

- Would give $\log n$ representation if we have a complete binary tree.
- If we cannot find an equipartition.
- If there is an equipartition, searching for S_1 and S_2 could be tiresome.

- Huffman encoding:

Fundamental idea: Least freq. elements placed far away from the root.

Most freq. are closer to root.

S: set of letters

P: List of freq. of letters in S

Algo (S, P):

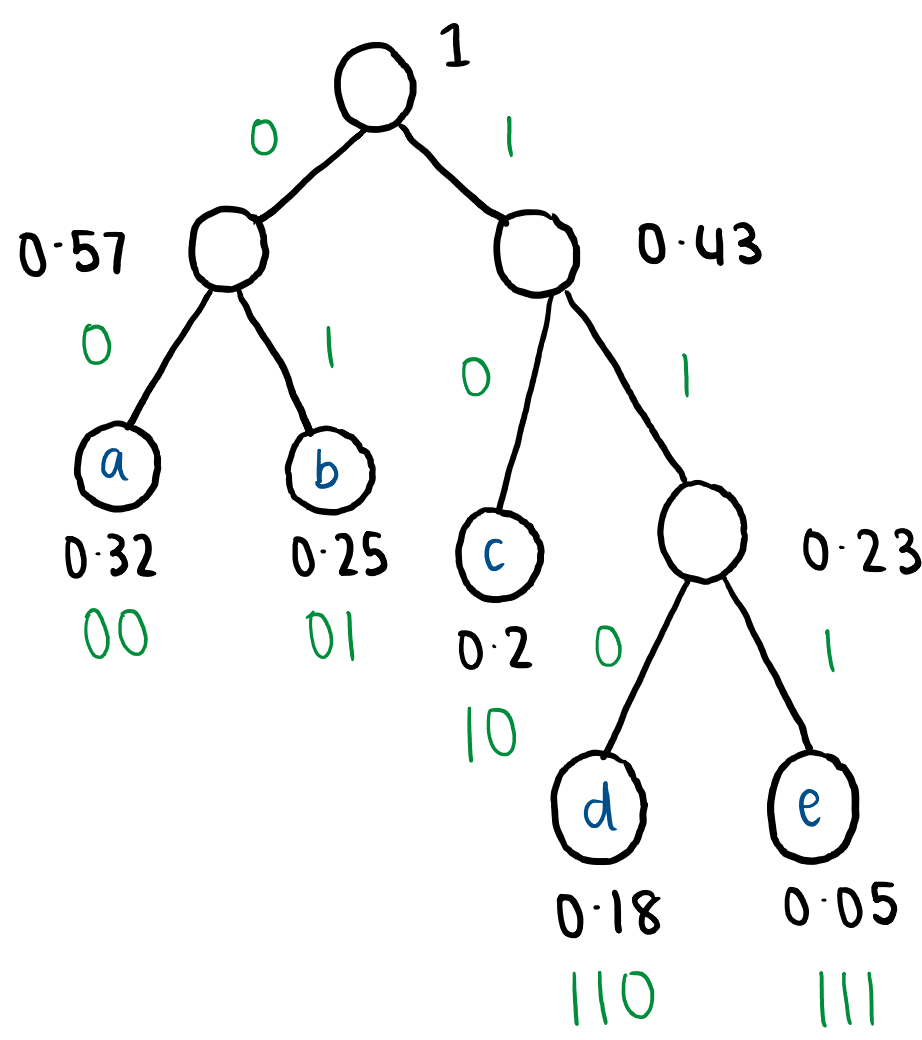
- Handle base cases
- Pick two least freq. elements in S (say 'y' and 'z').
- Remove y and z from S and P .
- Add a new letter w to S such that:

$$\text{freq}(w) = \text{freq}(y) + \text{freq}(z).$$
- Call updated lists S' and P' .
- Tree $T' \leftarrow \text{Algo}(S', P')$.
- Obtain a tree T by replacing the leaf w by a node with children y and z.

Return T.

e.g.

Letters	a	b	c	d	e	
Frequencies	0.32	0.25	0.2	0.18	0.05	Algo (S, P)
P ₁	0.32	0.25	0.2	0.23		Algo (S ₁ , P ₁)
P ₂	0.32	0.25	0.43			Algo (S ₂ , P ₂)
P ₃	0.57		0.43			Algo (S ₃ , P ₃)
P ₄						1



- $ABL = 2 \times 0.32 + 2 \times 0.25$
 $+ 2 \times 0.2 + 3 \times 0.18 + 3 \times 0.05$
 $= \underline{\underline{2.23}}$ better than 3
 $\lceil \log_2 5 \rceil = 3$